

Date.....

* Exception Handling :-

In professional API, it is crucial to move beyond basic responses (like just returning 200 OK or null) to properly inform the client of exactly what happened.

→ Components of an API Response :-

- Status Code → Communicates the outcome (e.g. success, failure, or client error).
- Headers → Provide metadata about the response.
- Body → Contains the actual data (mostly in JSON).

→ Why use standard API Response?

- Professionalism → A consistent format differentiates a production-grade API from a beginner project.
- Client Clarity → Proper HTTP status codes (like 200, 201, 204, 404, 500) help front-end developers handle various scenarios predictably.
- Global Handling → By avoiding try-catch blocks in every single controller method, you keep your code clean, readable and maintainable.

* ResponseEntity :-

The ResponseEntity class is the standard tool in Spring Boot to provide full control over the status, body, and headers of a response.

Instead of returning raw objects, wrap them in ResponseEntity to specify the HTTP status clearly.

→ Annotations :-

- @RestControllerAdvice → This marks a class as a global interceptor for all controllers. It allows you to handle exceptions thrown anywhere in your application.
- @ExceptionHandler → Placed on methods within the 'RestControllerAdvice' class, this annotation tells Spring Boot which specific exception class to catch and how to respond.

→ How to Implement?

- Create a new package (e.g. exception) and a class named 'GlobalExceptionHandler'.
- Define methods for different types of exceptions. For example, if a 'RuntimeException' occurs, the handler catches it and returns

a 'ResponseEntity' with a standardized status code like {500 Internal Server Error}.

* Creating Custom Exceptions :-

We can create custom exception classes that describe specific business logic failures.

→ Creating :-

- Create specialized exception classes (e.g. ResourceNotFound, DuplicateResource, etc.) by extending 'RuntimeException'.
- This allows your service layer to throw clear, descriptive errors that your 'GlobalExceptionHandler' can catch and map to specific HTTP status codes, such as "404 Not Found" or "409 Conflict".

→ Error Response DTO :-

- Returning raw strings is bad practice. Instead, define an 'ExceptionResponseDTO' class to ensure all errors follow a consistent structure.

→ Fields in DTO :-

- `timestamp` → When the error occurred.
- `statusCode` → The numeric HTTP status.
- `error` → A brief error category.
- `message` → A user-friendly explanation of the issue.
- `path` → The API endpoint that triggered the error.

* Validation Errors :-

- When using `@Valid` for input validation, catch "`MethodArgumentNotValidException`" in your "`GlobalExceptionHandler`".
- Extract field-specific errors using "`bindingResult.getFieldErrors()`" and map them into a map within your response DTO so the client knows exactly which fields failed validation.